Algorithm Analysis

CSALGCM

Prepared by: Austin Fernandez

Agenda

1. Review of CCDSALG
2. Asymptotic Analysis
3. Orders of Growth

Review for CCDSALG

```
float b, h, area;
scanf(“%f%f”,&b,&h);
area = b * h / 2;
printf(“Area = %f\n”,area);
```

```
int sum = 0, n;
scanf(“%d”,&n);
for(int i = 0; i <= n; i++){
   sum += i;
}
print(“sum = %d\n”,sum);
```

```
bool isComposite;
int n;
scanf(“%d”,&n);
if(n < 2){
   printf(“Neither prime nor composite”);
} else {
   isComposite = false;
   for(int i = 2; !isComposite && i < n; i++) {
      if(n % i == 0){
         isComposite = true;
      }
   }
   printf(“%s\n”,isComposite ? “Composite” : “Prime”);
}
```

```
int sum = 0, n;
scanf(“%d”,&n);
for(int i = 1; i < n; i++) {
   for(int j = 1; j < n; j++) {
      sum += i * j;
   }
}
printf(“Answer = %d\n”,sum);
```

```
int sum = 0, n;
scanf(“%d”,&n);
for(int i = 1; i < n; i++) {
   for(int j = 1; j <= i; j++) {
      sum += i * j;
   }
}
printf(“Answer = %d\n”,sum);
```

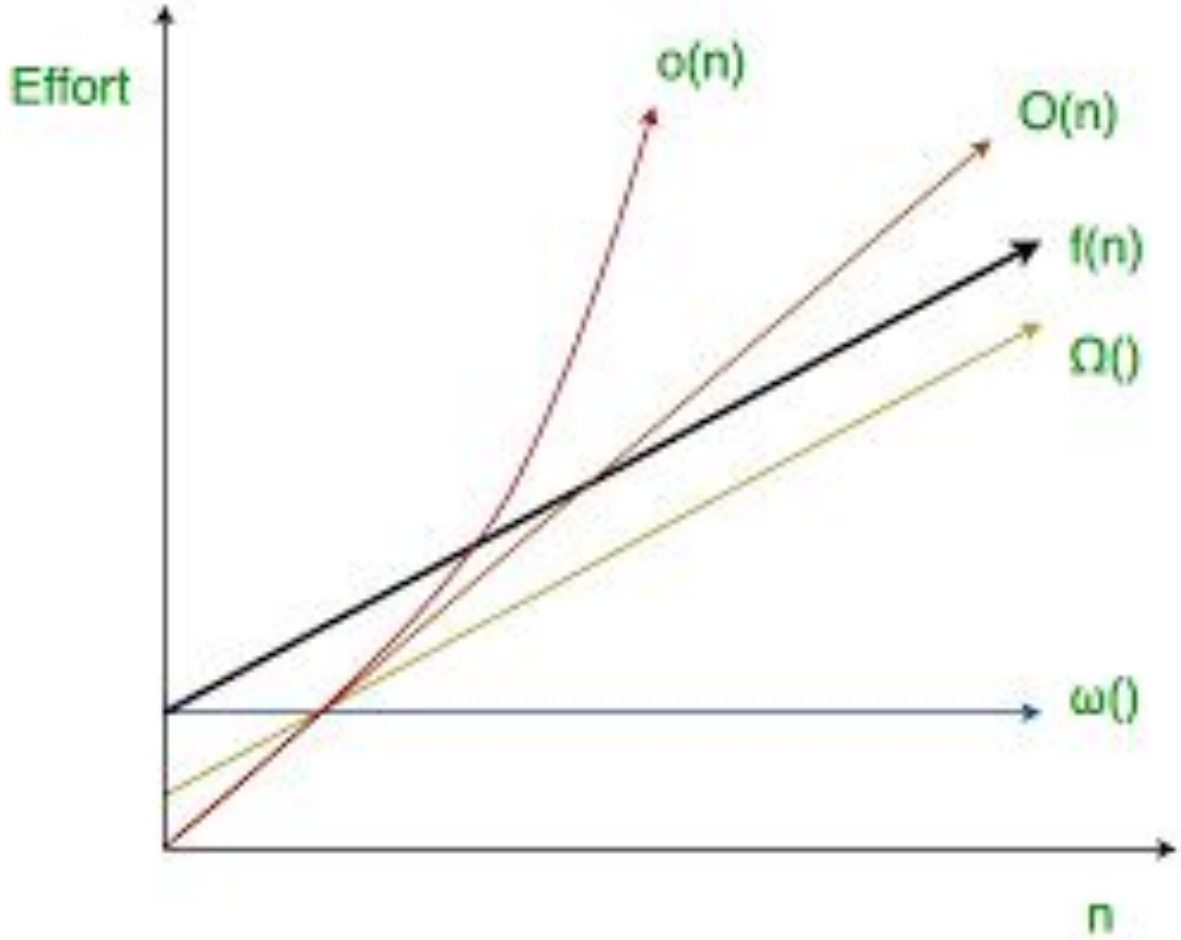
Asymptotic Analysis

Asymptotic Notation

- Used to describe the growth of a function.
- Loose Bounds:
  - Big-Omega : Ω(f(n))
  - Big-Theta : Θ(f(n))
  - Big-Oh : O(f(n))
- Tight Bounds:
  - Small-Omega: ω(f(n))
  - Small-Oh: o(f(n))

Asymptotic Notations

Notation	Layman's Explanation
$f(n) = \Omega(g(n))$	f(n) grows at least as fast as some order of g(n)
$f(n) = O(g(n))$	f(n) grows at worst as fast as some order of g(n)
$f(n) = \Theta(g(n))$	f(n) grows around as fast as some order of g(n)
$f(n) = o(g(n))$	f(n) grows strictly slower than some order of g(n)
$f(n) = \omega(g(n))$	f(n) grows strictly faster than some order of g(n)



Source: GeeksForGeeks

Loose Bounds

Notation	Formal Definition
$f(n) = \Omega(g(n))$	There exists real constants $n_0 >= 1$ and $c > 0$ such that $0 <= cg(n) <= f(n)$ for all $n >= n_0$
$f(n) = O(g(n))$	There exists real constants $n_0 >= 1$ and $c > 0$ such that $0 <= f(n) <= cg(n)$ for all $n >= n_0$
$f(n) = \Theta(g(n))$	There exists real constants $n_0 >= 1$ and $c_1, c_2 > 0$ such that $0 <= c_1 g(n) <= f(n) <= c_2 g(n)$ for all $n >= n_0$

Tight Bounds

Notation	Formal Definition
$f(n) = o(g(n))$	For all constants $c > 0$, there exists a constant $n_0 >= 1$ such that $0 <= f(n) < cg(n)$ for all $n >= n_0$
$f(n) = \omega(g(n))$	For all constants $c > 0$, there exists a constant $n_0 >= 1$ such that $0 <= cg(n) < f(n)$ for all $n >= n_0$

Loose Bounds (Limit Definition)

Notation	Formal Definition
f(n) = Ω(g(n))	$\lim_{n \to +\infty} \frac{f(n)}{g(n)} \neq 0$
f(n) = Θ(g(n))	$\lim_{n \to +\infty} \frac{f(n)}{g(n)} \neq 0, +\infty$
f(n) = O(g(n))	$\lim_{n \to +\infty} \frac{f(n)}{g(n)} \neq +\infty$

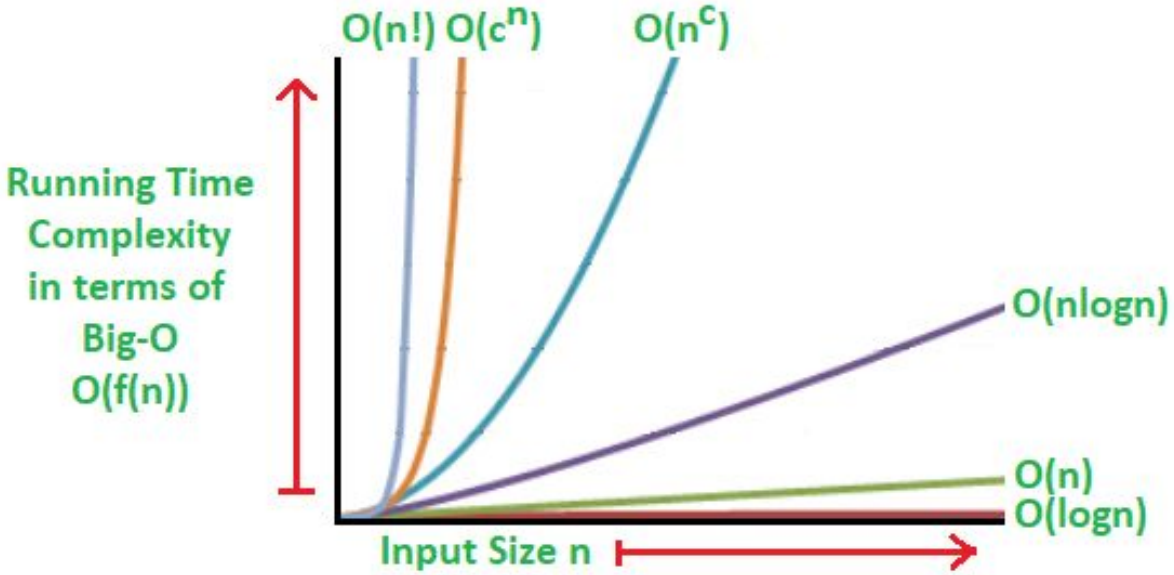
Tight Bounds (Limit Definition)

Notation	Formal Definition
f(n) = o(g(n))	$\lim_{n \to +\infty} \frac{f(n)}{g(n)} = 0$
f(n) = ω(g(n))	$\lim_{n \to +\infty} \frac{f(n)}{g(n)} = +\infty$

Orders of Growth

Commonly seen bounds

- $O(1)$
- $O(\log n)$
- $O(n)$
- $O(n \log n)$
- $O(n^2)$
- $O(n^3)$
- $O(2^n)$
- $O(n!)$



O(n!),O(c^n),O(n^c) - Worst

O(nlogn) - Bad

O(n) - Fair

O(logn) - Good

O(1) - Best

Source: GeeksForGeeks

Thank You!

References

Cormen, T. H., Leiserson, C. E., & Rivest, R. L., Stein, C. (2009). Introduction to Algorithms. 3rd Edition. MA: The MIT Press.

Dasgupta, S., Papadimitriou, C. H., & Vazirani, U. V. (2008). Algorithms. McGraw-Hill Higher Education.